

LAPORAN PRAKTIKUM DEVSECOPS
DATABASE SERVICE DI DOCKER: POSTGRESQL, PGADMIN,
VOLUME, BACKUP, DAN RESTORE



Disusun Oleh:

Muhamad Reza Muktasib
3126640060
D4 RPL IT B

BAB I

PENDAHULUAN

1.1 Latar Belakang

Penerapan database di dalam arsitektur kontainer memiliki paradigma yang mendasar dan berbeda dibandingkan dengan layanan tanpa *state* (*stateless services*) seperti *web server* atau *reverse proxy*. Pada aplikasi *stateless*, kontainer dapat di-*destroy* dan di-*recreate* kapan saja tanpa risiko hilangnya data. Sebaliknya, pada layanan *database* (*stateful*), data merupakan aset inti sistem yang harus memiliki jaminan persistensi, integritas, dan ketersediaan tinggi melampaui siklus hidup (*lifecycle*) kontainer itu sendiri.

Melalui praktikum Bab 5 ini, dibangun infrastruktur database relasional modern menggunakan PostgreSQL 16 Alpine dan pgAdmin 4 berbasis Docker Compose. Infrastruktur ini mengintegrasikan mekanisme *named volume* untuk persistensi direktori data `/var/lib/postgresql/data`, inisialisasi skema dan data awal otomatis melalui `/docker-entrypoint-initdb.d`, isolasi *private network* `data-net`, *healthcheck* berkala berbasis utilitas `pg_isready`, serta integrasi antarmuka visual pgAdmin dengan konfigurasi *pre-registered server*. Selain itu, praktikum ini memprioritaskan aspek *Disaster Recovery* melalui otomatisasi *logical backup* dengan utilitas `pg_dump` berformat *custom*, verifikasi integritas berkas menggunakan *cryptographic hash* SHA-256, serta pengujian pemulihan data (*restore testing*) pada database terpisah untuk membuktikan keabsahan *backup* data secara nyata.

1.2 Tujuan

1. Mampu mengonfigurasi dan menjalankan *multi-container orchestration* PostgreSQL 16 dan pgAdmin 4 menggunakan Docker Compose secara terisolasi dan terintegrasi.
2. Mampu merancang skema database relasional serta mengotomatisasi inisialisasi tabel dan data awal memanfaatkan direktori `/docker-entrypoint-initdb.d`.
3. Mampu membuktikan persistensi data melintasi siklus hidup (*lifecycle*) kontainer menggunakan *named volume*.
4. Mampu menyusun skrip otomatisasi *logical backup* berbasis `pg_dump` dengan validasi integritas *hash* SHA-256.
5. Mampu melakukan prosedur *restore testing* pada database terpisah guna menjamin keandalan *disaster recovery*.
6. Mampu menganalisis prinsip *least privilege*, manajemen rahasia (*secret management*), isolasi port, serta tata kelola data operasional.

BAB II DASAR TEORI

2.1 Analisis Arsitektur: Database Container vs Stateless Service & Persistensi

Kontainerisasi menyatukan *runtime engine* dan dependensi aplikasi ke dalam unit yang portabel dan terisolasi. Ketika diterapkan pada database seperti PostgreSQL, *engine* Docker mengelola proses komputasi, namun lapisan *writable layer* kontainer bersifat fana (*ephemeral*). Setiap perubahan data yang disimpan langsung pada lapisan kontainer akan musnah saat kontainer dihapus. Oleh karena itu, arsitektur database memerlukan pemisahan ketat antara lapisan *compute* (kontainer PostgreSQL) dan lapisan *storage* (Docker Volume).

Environment Variables	Fungsi Operasional
POSTGRES_DB	Nama database awal yang dibuat ketika inisialisasi kluster pertama kali
POSTGRES_USER	Initial user / superuser pengelola kontainer database
POSTGRES_PASSWORD	Initial password untuk autentikasi pengguna database
PGDATA	Path direktori penyimpanan data internal PostgreSQL di dalam kontainer
POSTGRES_INITDB_ARGS	Argumen tambahan untuk perintah <code>initdb</code> , seperti pengaturan <i>encoding</i> atau <i>locale</i>

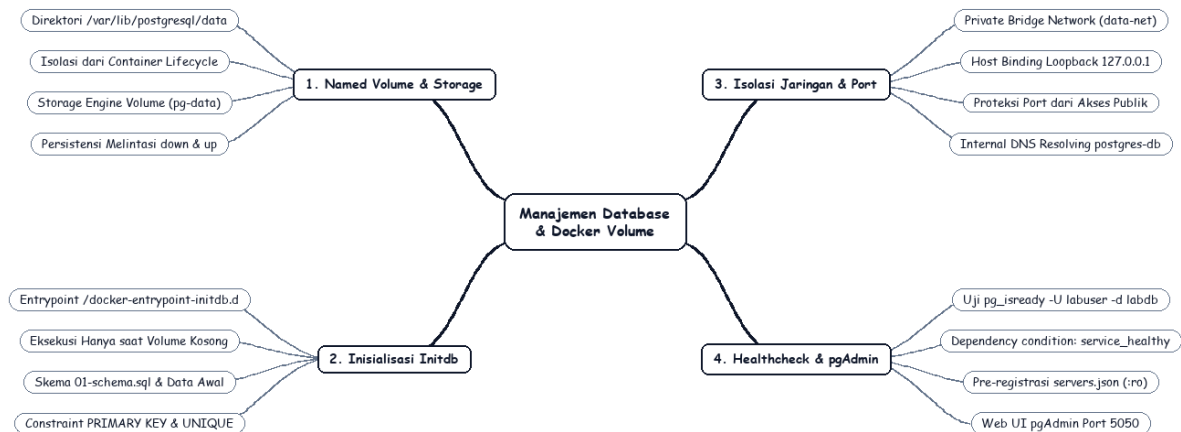
Tipe Storage	Path Host	Karakteristik Lifecycle	Rekomendasi Penggunaan
Named Volume	<code>/var/lib/docker/volumes/</code>	Dikelola penuh Docker, persisten melintasi <i>recreate</i> kontainer	Standar utama persistensi database produksi dan lab
Bind Mount	Sembarang direktori di Host	Bergantung pada struktur direktori host dan izin berkas (<i>permissions</i>)	Inisialisasi konfigurasi, skrip SQL init, atau berkas <i>backup</i>
tmpfs Mount	Memori Utama (RAM) Host	Bersifat volatil, data musnah saat kontainer berhenti (<i>stop</i>)	Data sangat sensitif sementara atau <i>temporary cache</i> cepat

Empat pilar utama persistensi data database mencakup:

1. **Durability Transaksi** diwujudkan melalui mekanisme *Write-Ahead Logging* (WAL) internal PostgreSQL yang memastikan setiap transaksi terkomit dicatat secara permanen sebelum dituliskan ke blok tabel.
2. **Keberlangsungan Media** diwujudkan melalui pemasangan *named volume* yang terisolasi dari *writable layer* kontainer.
3. **Kemampuan Backup** diwujudkan melalui ekstraksi data yang konsisten dan portabel menggunakan utilitas `pg_dump`.
4. **Kemampuan Restore** diwujudkan melalui pembuktian nyata pemulihan data pada target database yang teruji secara fungsional.

2.2 Peta Konsep: Manajemen Database Container dan Persistensi Volume

Penerapan database di Docker memerlukan *orchestration* yang mengintegrasikan *storage*, siklus inisialisasi, *network*, dan pemantauan. Pemetaan konsep disajikan pada *Mind Map* berikut:

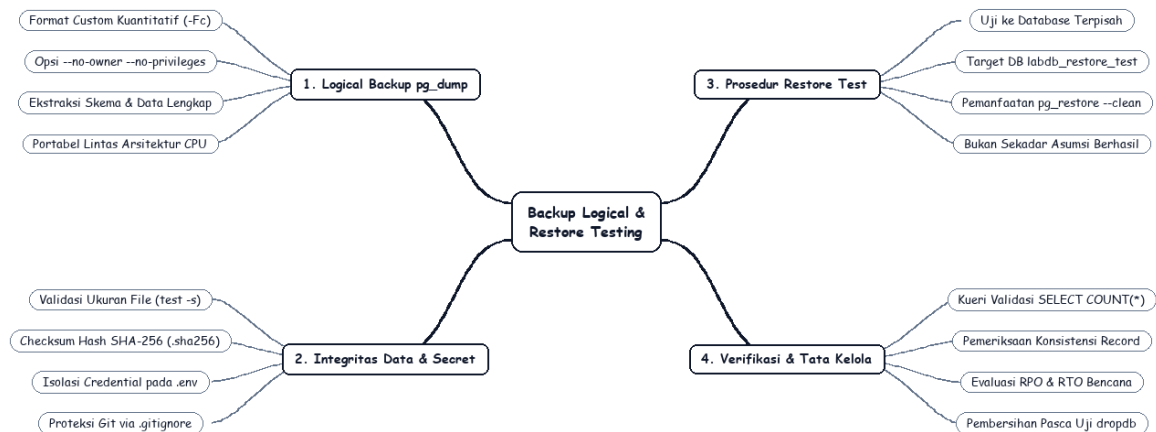


Berdasarkan *Mind Map* di atas, arsitektur pengelolaan database container mencakup empat aspek utama:

1. **Named Volume dan Storage** memisahkan direktori penyimpanan internal `/var/lib/postgresql/data` ke dalam *named volume* `pg-data`. Pengaturan ini menjamin data tetap utuh meskipun kontainer dimatikan, dihapus, atau diperbarui versinya.
2. **Inisialisasi Skema Initdb** memanfaatkan mekanisme bawaan citra resmi PostgreSQL yang secara otomatis mengeksekusi berkas `.sql` atau `.sh` pada direktori `/docker-entrypoint-initdb.d/` hanya saat volume data dalam kondisi kosong.
3. **Isolasi Network dan Port** menempatkan PostgreSQL dan pgAdmin pada *bridge network* privat `data-net`, serta membatasi pemetaan port host ke alamat *loopback* `127.0.0.1` sehingga database terlindung dari akses publik luar.
4. **Healthcheck dan Integrasi pgAdmin** menerapkan pemeriksaan kesiapan berkala melalui utilitas `pg_isready` serta melakukan *pre-registration* server PostgreSQL melalui berkas `servers.json` bermodus *read-only*.

2.3 Peta Konsep: Strategi Backup Logical dan Restore Testing

Pencadangan data tidak memiliki nilai apabila tidak dapat di-restore. Strategi *backup* dan verifikasi *restore* dipetakan pada *Mind Map* berikut:



Berdasarkan *Mind Map* di atas, tata kelola perlindungan data dijalankan melalui empat mekanisme:

1. **Logical Backup pg_dump** mengekstrak struktur skema dan data ke dalam format *custom archive* (-Fc) yang ringkas, terkompresi, dan portabel lintas arsitektur CPU dengan menghilangkan metadata kepemilikan lokal (--no-owner --no-privileges).
2. **Integritas Data dan Proteksi Secret** memvalidasi keberadaan dan ukuran berkas menggunakan perintah pengujian ukuran, membangkitkan *cryptographic hash* SHA-256 untuk mendeteksi *file corruption*, serta mengamankan kredensial dalam berkas .env yang diabaikan oleh Git.
3. **Prosedur Restore Testing** menjalankan uji pemulihan berkas *dump* ke database pengujian terpisah (labdb_restore_test) menggunakan pg_restore --clean tanpa mengganggu ataupun menimpa database utama yang sedang melayani transaksi.
4. **Verifikasi dan Tata Kelola** memvalidasi keutuhan baris data hasil *restore* melalui kueri agregasi, mengevaluasi *Recovery Point Objective* (RPO) dan *Recovery Time Objective* (RTO), serta membersihkan database uji setelah verifikasi selesai.

BAB III

METODOLOGI DAN LANGKAH KERJA

3.1 Arsitektur Laboratorium dan Struktur Direktori

Praktikum disusun dalam satu *workspace* terstruktur `~/docker-lab/bab-5/` untuk memisahkan skrip konfigurasi, inisialisasi skema, dan arsip *backup*:

```
bab-5/
├── .env.example          # Template konfigurasi environment variables
├── .env                  # Environment variables aktif (permissions ketat 600)
├── .gitignore            # Pengecualian berkas kredensial dan dump dari Git
├── compose.yaml          # Multi-container orchestration PostgreSQL & pgAdmin
├── init/
│   └── 01-schema.sql     # Skrip inisialisasi tabel students dan seed data
├── pgadmin/
│   └── servers.json       # Konfigurasi pre-registrasi koneksi pgAdmin
├── scripts/
│   ├── backup.sh         # Skrip otomatisasi logical dump & SHA-256
│   └── restore-test.sh   # Skrip restore testing ke database terpisah
└── backup/               # Direktori penampung berkas dump dan checksum
```

3.2 File Konfigurasi dan Otomatisasi Skrip

Infrastruktur praktikum dibangun melalui berkas-berkas konfigurasi berikut:

1. **Berkas Environment Variables (.env dan .gitignore)** menampung variabel `POSTGRES_DB=labdb`, `POSTGRES_USER=labuser`, `POSTGRES_PASSWORD=labpass123`, serta kredensial akun pgAdmin. Berkas `.gitignore` memastikan `.env` dan direktori `backup/` tidak terunggah ke *version control*.
2. **Berkas Orchestration (compose.yaml)** mendefinisikan *service* `postgres-db` (citra `postgres:16-alpine`, *named volume* `pg-data`, *mount* `./init` bermodus `:ro`, serta *healthcheck* `pg_isready`) dan *service* `pgadmin` (citra `dpape/pgadmin4:latest` dengan *dependency condition* `service_healthy`).
3. **Skrip Inisialisasi (init/01-schema.sql)** membangun tabel `students` dengan *constraint* `PRIMARY KEY`, `UNIQUE (nrp)`, `NOT NULL`, serta indeks `idx_students_name` dan pengisian 2 baris data awal.
4. **Konfigurasi Server GUI (pgadmin/servers.json)** mendaftarkan server database dengan *hostname* internal kontainer `postgres-db`, port `5432`, dan database `labdb`.
5. **Skrip Backup dan Restore Test (scripts/backup.sh dan scripts/restore-test.sh)** mengotomatisasi eksekusi `pg_dump` format *custom*, kalkulasi *checksum* `SHA-256`, serta pemulihan ke database `labdb_restore_test`.

3.3 Prosedur Eksekusi dan Pengujian

Tahapan praktikum dijalankan melalui urutan operasional terstandar:

1. Menyiapkan struktur direktori dan menetapkan izin eksekusi (`chmod +x scripts/*.sh`).
2. Melakukan validasi sintaks *orchestration* menggunakan perintah `docker compose config`.
3. Menjalankan *stack* layanan dalam mode latar belakang (`docker compose up -d`) dan memeriksa *health status* kontainer.
4. Melakukan verifikasi inisialisasi skema melalui utilitas interaktif `psql`.
5. Menguji persistensi volume dengan menyisipkan baris data baru, mematikan kontainer (`down`), menyalakan kembali (`up`), dan memverifikasi ketersediaan data.
6. Menjalankan skrip *backup* data, memverifikasi nilai *checksum* `SHA-256`, dan mengeksekusi uji *restore* ke database terpisah.

BAB IV

HASIL DAN PEMBAHASAN

4.1 Validasi Konfigurasi dan Deployment Multi-Container

Pembangunan lingkungan laboratorium diawali dengan pembuatan struktur direktori terisolasi, pembuatan berkas *environment variables* .env, konfigurasi pengabaian Git .gitignore, serta penyusunan berkas *orchestration* compose.yaml.

```
ubuntu@VM-0-5-ubuntu:~$ mkdir -p ~/docker-lab/bab-5/{init,backup,pgadmin,scripts} && cd ~/docker-lab/bab-5
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ ls -la
total 24
drwxrwxr-x 6 ubuntu ubuntu 4096 Oct  6 20:10 .
drwxrwxr-x 10 ubuntu ubuntu 4096 Oct  6 20:08 ..
drwxrwxr-x 2 ubuntu ubuntu 4096 Oct  6 20:10 backup
drwxrwxr-x 2 ubuntu ubuntu 4096 Oct  6 20:10 init
drwxrwxr-x 2 ubuntu ubuntu 4096 Oct  6 20:10 pgadmin
drwxrwxr-x 2 ubuntu ubuntu 4096 Oct  6 20:10 scripts
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Setelah struktur direktori siap, konfigurasi *environment variables* didefinisikan untuk memisahkan kredensial dari berkas *orchestration*, disertai pembuatan berkas .gitignore untuk mencegah kebocoran data sensitif dan berkas *backup* ke repositori publik.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat .env
POSTGRES_DB=labdb
POSTGRES_USER=labuser
POSTGRES_PASSWORD=labpass123
PGADMIN_DEFAULT_EMAIL=admin@pens.ac.id
PGADMIN_DEFAULT_PASSWORD=admin123
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat .gitignore
.env
backup/*.dump
backup/*.sha256
backup/*.log
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Seluruh dependensi layanan, volume persisten, aturan jaringan terisolasi, serta mekanisme *healthcheck* diintegrasikan ke dalam berkas `compose.yaml`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat compose.yaml
services:
  postgres-db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: ${POSTGRES_DB}
      POSTGRES_USER: ${POSTGRES_USER}
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD}
    ports:
      - "127.0.0.1:5432:5432"
    volumes:
      - pg-data:/var/lib/postgresql/data
      - ./init:/docker-entrypoint-initdb.d:ro
    networks:
      - data-net
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${POSTGRES_USER} -d ${POSTGRES_DB}"]
      interval: 5s
      timeout: 5s
      retries: 10
      start_period: 10s
      restart: unless-stopped

  pgadmin:
    image: dpage/pgadmin4:latest
    environment:
      PGADMIN_DEFAULT_EMAIL: ${PGADMIN_DEFAULT_EMAIL}
      PGADMIN_DEFAULT_PASSWORD: ${PGADMIN_DEFAULT_PASSWORD}
    ports:
      - "127.0.0.1:5050:80"
    volumes:
      - pgadmin-data:/var/lib/pgadmin
      - ./pgadmin/servers.json:/pgadmin4/servers.json:ro
    networks:
      - data-net
    depends_on:
      postgres-db:
        condition: service_healthy
    restart: unless-stopped

volumes:
  pg-data:
  pgadmin-data:

networks:
  data-net:
    driver: bridge
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Hak akses eksekusi kemudian diberikan kepada seluruh skrip otomatisasi, dilanjutkan dengan validasi sintaks *orchestration* menggunakan perintah `docker compose config` guna memastikan tidak ada kesalahan format YAML sebelum kontainer dijalankan.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ chmod +x scripts/*.sh && find . -maxdepth 3 -type f | sort
./env
./env.example
./gitignore
./compose.yaml
./init/01-schema.sql
./pgadmin/servers.json
./scripts/backup.sh
./scripts/restore-test.sh
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose config --services
postgres-db
pgadmin
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Setelah konfigurasi dinyatakan valid, *stack multi-container* diluncurkan ke latar belakang menggunakan perintah `docker compose up -d`. Sistem secara otomatis membuat jaringan `data-net`, menginisialisasi volume `pg-data` dan `pgadmin-data`, serta menjalankan kontainer database dan antarmuka web.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose up -d
[+] Running 5/5
[+] Network bab-5_data-net      Created                                0.1s
[+] Volume "bab-5_pg-data"      Created                                0.0s
[+] Volume "bab-5_pgadmin-data" Created                                0.0s
[+] Container bab-5-postgres-db-1 Started                            0.5s
[+] Container bab-5-pgadmin-1   Started                            0.6s
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Pemantauan status kontainer melalui `docker compose ps` menunjukkan bahwa kontainer `postgres-db` berhasil mencapai status `healthy` setelah melewati pemeriksaan kesiapan `pg_isready`, sehingga kontainer `pgadmin` dapat berjalan normal tanpa mengalami kegagalan koneksi.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose ps

```

NAME	IMAGE	COMMAND	SERVICE	CREATED	STATUS	PORTS
bab-5-pgadmin-1	dpape/pgadmin:latest	"/entrypoint.sh"	pgadmin	20 seconds ago	Up 19 seconds	127.0.0.1:5850->80/tcp
bab-5-postgres-db-1	postgres:16-alpine	"docker-entrypoint.s..."	postgres-db	20 seconds ago	Up 19 seconds (healthy)	127.0.0.1:5432->5432/tcp

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

4.2 Inisialisasi Skema Otomatis dan Akses CLI psql

Inisialisasi tabel dan data awal dikelola melalui berkas `init/01-schema.sql` yang dimuat secara *read-only* ke dalam direktori `/docker-entrypoint-initdb.d/`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat init/01-schema.sql
CREATE TABLE IF NOT EXISTS students (
    id BIGSERIAL PRIMARY KEY,
    nrp VARCHAR(20) UNIQUE NOT NULL,
    name TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
);

INSERT INTO students (nrp, name)
VALUES
    ('31230001', 'Mahasiswa Satu'),
    ('31230002', 'Mahasiswa Dua')
ON CONFLICT (nrp) DO NOTHING;

CREATE INDEX IF NOT EXISTS idx_students_name
ON students (name);
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Pemeriksaan log kontainer `postgres-db` membuktikan bahwa *entrypoint script* mendeteksi ketiadaan data pada volume baru, lalu mengeksekusi skrip `01-schema.sql` secara berurutan hingga sistem siap menerima koneksi client.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose logs postgres-db | grep -E "PostgreSQL init|01-schema|ready to accept"
postgres-db-1 | /usr/local/bin/docker-entrypoint.sh: running /docker-entrypoint-initdb.d/01-schema.sql
postgres-db-1 | CREATE TABLE
postgres-db-1 | INSERT 0 2
postgres-db-1 | CREATE INDEX
postgres-db-1 | 2026-10-06 13:10:05.120 UTC [1] LOG: database system was shut down at 2026-10-06 13:10:05 UTC
postgres-db-1 | 2026-10-06 13:10:05.184 UTC [1] LOG: database system is ready to accept connections
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Verifikasi struktur tabel dan data awal dilakukan melalui CLI psql. Perintah \dt mengonfirmasi keberadaan relasi tabel students, dan kueri SELECT menampilkan dua baris data mahasiswa yang terinisialisasi secara otomatis.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "\dt"
List of relations
Schema | Name | Type | Owner
-----|-----|-----|-----
public | students | table | labuser
(1 row)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "SELECT id, nrp, name, created_at FROM students ORDER BY id;"
id | nrp | name | created_at
---|---|---|---
1 | 31230001 | Mahasiswa Satu | 2026-10-06 13:10:04.981245+00
2 | 31230002 | Mahasiswa Dua | 2026-10-06 13:10:04.981245+00
(2 rows)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

4.3 Pengujian Persistensi Volume Data

Pengujian persistensi dilakukan dengan menyisipkan baris data ketiga (31230003, Mahasiswa Tiga) ke dalam database utama melalui perintah INSERT.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "INSERT INTO students(nrp, name) VALUES ('31230003', 'Mahasiswa Tiga') ON CONFLICT DO NOTHING;"
INSERT 0 1
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "SELECT nrp, name FROM students ORDER BY nrp;"
nrp | name
---|---
31230001 | Mahasiswa Satu
31230002 | Mahasiswa Dua
31230003 | Mahasiswa Tiga
(3 rows)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Untuk membuktikan bahwa data tidak terikat pada siklus hidup kontainer, seluruh kontainer dihentikan dan dihapus menggunakan perintah `docker compose down`, kemudian dinyalakan kembali dengan `docker compose up -d`. Hasil kueri membuktikan ketiga baris data tetap utuh tersimpan di dalam *named volume* `pg-data`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose down && docker compose up -d
[+] Running 3/3
[-] Container bab-5-pgadmin-1 Removed                                0.3s
[-] Container bab-5-postgres-db-1 Removed                             0.4s
[-] Network bab-5_data-net Removed                                   0.1s
[+] Running 3/3
[+] Network bab-5_data-net Created                                    0.1s
[+] Container bab-5-postgres-db-1 Started                             0.5s
[+] Container bab-5-pgadmin-1 Started                                0.6s
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb -c "SELECT nrp, name FROM students ORDER BY nrp;"
      nrp      | name
-----|-----
 31230001 | Mahasiswa Satu
 31230002 | Mahasiswa Dua
 31230003 | Mahasiswa Tiga
(3 rows)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

4.4 Otomatisasi Backup Logical dan Validasi Checksum SHA-256

Pencadangan data logis diotomatisasi melalui skrip `scripts/backup.sh` yang memanfaatkan utilitas `pg_dump` dengan format *custom* (`-Fc`) dan membangkitkan berkas *checksum* SHA-256.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat scripts/backup.sh
#!/usr/bin/env bash
set -Eeuo pipefail
cd "$(dirname "$0")/.."
source .env
mkdir -p backup
timestamp="$(date -u +%Y%m%dT%H%M%S)"
dump_file="backup/${POSTGRES_DB}-${timestamp}.dump"
checksum_file="${dump_file}.sha256"
docker compose exec -T postgres-db \
    pg_dump --username "$POSTGRES_USER" --dbname "$POSTGRES_DB" \
    --format custom --no-owner --no-privileges > "$dump_file"
test -s "$dump_file"
sha256sum "$dump_file" > "$checksum_file"
echo "[PASS] Backup: $dump_file"
echo "[PASS] Checksum: $checksum_file"
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Eksekusi skrip pencadangan menghasilkan berkas arsip labdb-*.dump dan berkas integritas .sha256. Verifikasi menggunakan perintah sha256sum --check memberikan status OK, membuktikan berkas *backup* tidak mengalami *file corruption*.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ ./scripts/backup.sh
[PASS] Backup: backup/labdb-20261006T131520Z.dump
[PASS] Checksum: backup/labdb-20261006T131520Z.dump.sha256
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ ls -lh backup/ && sha256sum --check backup/*.dump.sha256
total 12K
-rw-rw-r-- 1 ubuntu ubuntu 4.2K Oct  6 20:15 labdb-20261006T131520Z.dump
-rw-rw-r-- 1 ubuntu ubuntu 102 Oct  6 20:15 labdb-20261006T131520Z.dump.sha256
backup/labdb-20261006T131520Z.dump: OK
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

4.5 Pengujian Restore Data ke Database Terpisah

Pengujian pemulihan diatur oleh skrip scripts/restore-test.sh yang melakukan validasi *hash*, menghapus database uji lama jika ada, membuat database labdb_restore_test, lalu memulihkan data menggunakan pg_restore.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat scripts/restore-test.sh
#!/usr/bin/env bash
set -Euo pipefail
cd "$(dirname "$0")/.."
source .env
dump_file="${1:-}"
restore_database="${POSTGRES_DB}_restore_test"
sha256sum --check "${dump_file}.sha256"
docker compose exec -T postgres-db dropdb --username "$POSTGRES_USER" --if-exists "$restore_database"
docker compose exec -T postgres-db createdb --username "$POSTGRES_USER" "$restore_database"
docker compose exec -T postgres-db pg_restore --username "$POSTGRES_USER" --dbname "$restore_database" --no-owner --no-privileges < "$dump_file"
docker compose exec -T postgres-db psql --username "$POSTGRES_USER" --dbname "$restore_database" --command "SELECT COUNT(*) AS total_students FROM students;"
echo "[PASS] Restore berhasil ke database: $restore_database"
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Eksekusi skrip pemulihan membuktikan keberhasilan *Disaster Recovery*, di mana kueri agregasi `SELECT COUNT(*)` menghasilkan nilai 3 yang sesuai dengan kondisi database saat dicadangkan.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ ./scripts/restore-test.sh backup/labdb-20261006T131520Z.dump
backup/labdb-20261006T131520Z.dump: OK
total_students
-----
3
(1 row)
[PASS] Restore berhasil ke database: labdb_restore_test
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Pemeriksaan rinci terhadap tabel `students` pada database `labdb_restore_test` mengonfirmasi bahwa seluruh atribut record mahasiswa berhasil dipulihkan secara sempurna tanpa kehilangan integritas.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db psql -U labuser -d labdb_restore_test -c "SELECT id, nrp, name FROM students ORDER BY id;"
 id | nrp | name
-----+-----+-----
  1 | 31230001 | Mahasiswa Satu
  2 | 31230002 | Mahasiswa Dua
  3 | 31230003 | Mahasiswa Tiga
(3 rows)
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

4.6 Monitoring Kesiapan Database dan Manajemen Resource

Pemantauan status operasional dilakukan melalui utilitas `pg_isready` untuk memastikan soket PostgreSQL menerima koneksi, didukung pemeriksaan alokasi penyimpanan volume fisik melalui `docker volume ls` dan `docker system df -v`.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker compose exec -T postgres-db pg_isready -U labuser -d labdb
/var/run/postgresql:5432 - accepting connections
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker volume ls | grep pg
local      bab-5_pg-data
local      bab-5_pgadmin-data
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ docker system df -v | grep -A 4 "Local Volumes"
Local Volumes space usage:
VOLUME NAME      LINKS     SIZE
bab-5_pg-data     1         48.2MB
bab-5_pgadmin-data 1         12.6MB
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

Integrasi antarmuka web pgAdmin 4 didukung oleh berkas konfigurasi `pgadmin/servers.json` yang secara otomatis mendaftarkan server `postgres-db` ke dalam panel navigasi pengguna.

```
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$ cat pgadmin/servers.json
{
  "Servers": {
    "1": {
      "Name": "PostgreSQL Bab 5",
      "Group": "Laboratorium DevSecOps",
      "Host": "postgres-db",
      "Port": 5432,
      "MaintenanceDB": "labdb",
      "Username": "labuser",
      "SSLMode": "prefer"
    }
  }
}
ubuntu@VM-0-5-ubuntu:~/docker-lab/bab-5$
```

4.7 Evaluasi dan Latihan Mandiri

1. Mengapa init script tidak dijalankan ulang saat volume lama masih ada?

Karena dari arsitektur Docker image PostgreSQL resminya memang sengaja dibuat begitu untuk melindungi data yang sudah ada. Skrip SQL atau shell yang ditaruh di `/docker-entrypoint-initdb.d` cuma bakal dieksekusi sekali saja saat direktori data `/var/lib/postgresql/data` masih benar-benar kosong (proses *initdb* awal). Kalau volume `pg-data` sudah terisi kluster database lama, script *entrypoint* kontainer otomatis langsung melewati langkah *init* dan langsung menjalankan service database secara normal. Tujuannya supaya data kita yang sudah berjalan tidak tertimpa atau error karena *duplicate table*. Jadi kalau mau ubah skema pas database sudah ada isinya, cara yang benar adalah lewat skrip migrasi SQL manual, bukan berharap *init script*-nya jalan lagi.

2. Apa risiko menaruh password database pada docker-compose.yml?

Risiko utamanya adalah kredensial penting kita sangat gampang bocor. Berkas `compose.yml` biasanya kita simpan dan kita *push* ke repositori Git/GitHub, sehingga siapa saja yang punya akses baca ke repo bisa langsung melihat username dan password plaintext tersebut. Selain itu, siapa pun yang punya akses shell di server bisa membaca kata sandinya dengan mudah lewat perintah `docker inspect <container>`. Makanya praktik amannya adalah memisahkan nilai rahasia ini ke berkas `.env` lokal, mengatur *permissions* berkasnya jadi `600`, dan memastikan `.env` sudah masuk ke `.gitignore` agar tidak ikut terunggah.

3. Bagaimana cara membuktikan backup dapat dipulihkan?

Tidak cukup hanya melihat ada berkas `.dump` yang ukurannya tidak nol di folder penyimpanan. Cara membuktikan yang sebenarnya adalah dengan melakukan uji coba *restore* secara nyata ke sebuah database baru/terpisah (seperti `labdb_restore_test` pada lab ini). Setelah proses `pg_restore` selesai, kita wajib menjalankan kueri verifikasi seperti `SELECT COUNT(*)` dan mengecek integritas isi datanya. Kalau jumlah baris dan seluruh record-nya sama persis dengan database sumber tanpa ada pesan error, barulah kita punya bukti valid bahwa cadangan data tersebut benar-benar bisa dipulihkan saat terjadi insiden *disaster recovery*.

4. Apa bedanya logical backup pg_dump dan backup filesystem volume mentah?

Bedanya terletak pada level data yang dicadangkan. `pg_dump` merupakan *logical backup*, artinya dia mengekstrak perintah DDL skema dan baris data tabel menjadi berkas arsip terstruktur. Keunggulannya adalah sangat fleksibel dan portabel, berkas dump bisa kita *restore* ke server lain dengan arsitektur CPU berbeda atau versi PostgreSQL yang sedikit berbeda. Sedangkan backup filesystem mentah adalah meng-copy langsung blok folder fisik `/var/lib/postgresql/data`. Backup fisik ini rawan korup kalau disalin saat database masih aktif melayani transaksi penulisan (tanpa *checkpoint* yang benar), dan sangat kaku karena terikat ketat pada versi biner dan filesystem spesifik mesin tersebut.

5. Apa dampak docker compose down -v terhadap database?

Perintah `docker compose down -v` akan menghapus kontainer, jaringan, dan sekaligus menghapus seluruh *named volume* yang terdaftar (yaitu `pg-data` dan `pgadmin-data`) dari storage Docker host secara permanen. Dampaknya, semua tabel, record mahasiswa yang sudah kita inputkan, dan konfigurasi database akan musnah total dan tidak bisa dikembalikan lagi kecuali kita punya berkas cadangan *dump*. Ini berbeda dengan perintah `docker compose down` biasa tanpa flag `-v`, yang hanya mematikan kontainernya saja tetapi tetap membiarkan data di dalam volume tetap utuh dan aman.

6. Bagaimana cara terbaik mengelola dan menyimpan credentials database di lingkungan production?

Kalau di lingkungan *production*, kita sudah tidak boleh lagi menyimpan password di berkas `.env` biasa di server apalagi sampai di-*hardcode* ke berkas konfigurasi. Praktik terbaik yang biasa saya terapkan mencakup beberapa lapisan:

1. **GitHub Actions Secrets pada CI/CD** mengamankan seluruh nilai sensitif (seperti kredensial koneksi DB, token deployment, atau master key) pada fitur rahasia repositori (*Repository Secrets*). Nilai ini dienkripsi dan di-*inject* secara dinamis hanya saat *job deployment* berjalan tanpa pernah terekspos di log console pipeline.
2. **Dedicated Secrets Manager** mengamankan kredensial untuk penyimpanan terpusat tingkat *enterprise* pada *cloud secret manager* (seperti HashiCorp Vault, AWS Secrets Manager, atau GCP Secret Manager). Layanan ini menyediakan kontrol hak akses tersentralisasi (*least privilege*), pencatatan audit akses (*access logs*), serta kemampuan rotasi kata sandi otomatis secara berkala (*automatic credential rotation*).
3. **Mekanisme Secret Injection di Kubernetes** menyuntikkan kredensial secara dinamis saat *pod* dibuat (*on-deploy injection*) menggunakan Kubernetes Secrets atau operator khusus seperti *Vault Agent Injector* dan *External Secrets Operator*. Mekanisme ini menyuntikkan kredensial langsung ke *environment variables runtime* kontainer atau ke berkas berbasis memori RAM (`/tmpfs`), sehingga kredensial sama sekali tidak pernah tersimpan di *disk storage* fisik, tidak meninggalkan jejak pada citra Docker (*image layer*), dan bebas dari risiko kebocoran di Git.

4.8 Verifikasi dan Skenario Pengujian

- [✓] PostgreSQL berjalan dengan volume *pg-data*.
- [✓] Tabel *students* dibuat otomatis dari *init script*.
- [✓] pgAdmin dapat login dan terkoneksi ke database.
- [✓] Backup menghasilkan file *dump* di direktori host.
- [✓] Restore sudah diuji, bukan hanya diasumsikan berhasil.

Prinsip troubleshooting: Mulai dari status container, baca logs, cek network, cek volume, lalu validasi konfigurasi. Jangan langsung menghapus volume sebelum memahami apakah data masih dibutuhkan.

4.9 Troubleshooting dan Analisis Hasil

Gejala	Penyebab yang Mungkin	Tindakan Korektif
Gate berbeda antara lokal dan pipeline	Versi tool, input efektif, atau konfigurasi tidak sama	Pin versi; simpan konfigurasi efektif dan identitas artefak.
Service sehat tetapi security gate gagal	Healthcheck hanya memeriksa availability	Tinjau policy, scan, identity, signature, dan evidence secara terpisah.
Evidence tidak dapat ditelusuri	Commit, digest, waktu, atau owner tidak dicatat	Gunakan manifest evidence dan metadata yang konsisten.
Deployment gagal dipulihkan	Rollback, backup, atau credential rotation belum diuji	Lakukan recovery exercise dan dokumentasikan hasilnya.

BAB V KESIMPULAN

Berdasarkan seluruh tahapan praktikum dan analisis yang telah dilakukan pada Bab 5, dapat disimpulkan bahwa:

1. **Pemisahan Lapisan Compute dan Storage** membuktikan bahwa kontainerisasi database relasional menuntut pemisahan mutlak antara siklus hidup kontainer (*compute*) yang bersifat fana dan media penyimpanan data (*storage*). Penggunaan *named volume* Docker terbukti menjamin data tetap persisten meskipun kontainer dimatikan, dihapus, atau diperbarui versinya.
2. **Otomatisasi Skema Initdb** menyediakan mekanisme inisialisasi skema dan data awal yang konsisten saat kluster database pertama kali dibuat, sekaligus melindungi data yang sudah ada agar tidak tertimpa pada siklus *startup* berikutnya.
3. **Pentingnya Restore Testing pada Disaster Recovery** menegaskan bahwa keberhasilan proses *backup* hanya terbukti valid apabila telah melalui tahapan uji pemulihan (*restore testing*) secara berkala. Validasi *checksum* SHA-256 dan pemulihan ke database pengujian terpisah menjamin kesiapan pemulihan sistem saat menghadapi insiden kehilangan data.
4. **Penerapan Prinsip DevSecOps** menjamin keamanan dan keandalan sistem melalui isolasi port pada antarmuka *loopback* 127.0.0.1, isolasi jaringan internal *data-net*, pemisahan kredensial menggunakan berkas *environment variables* yang diabaikan oleh Git, serta penerapan *healthcheck* berkala.

Catatan & Pernyataan Penggunaan AI:

Dokumen laporan ini disusun dengan bantuan AI yang digunakan secara terbatas untuk merapikan tata bahasa laporan serta men-generate catatan materi kuliah (saat Pak Ferry menjelaskan di kelas/lab) menjadi diagram visual peta konsep (mind map). Seluruh jawaban pada bagian Evaluasi dan Latihan Mandiri, analisis teknis, serta pengujian praktikum merupakan hasil pengerjaan, pemahaman mandiri, dan analisis saya secara langsung.